Experiment No: 3

Student Name: Prionti Das	**UID:** 24BAI71005
Branch: AIT-CSE	**Section/Group:** 24AIT_NTPP 3 [B]
Semester: 5TH	**Date of Performance:**
Subject Name: Competitive Coding – II	**Subject Code:** 24CSP-305

Experiment 3.1

Aim: To implement a **Queue data structure using two stacks** in C++.

Objective:

- To understand the working of a Queue using **FIFO** principle.
- To implement queue operations using stacks.
- To understand the use of two stacks for reversing element order.

Problem Statement / LeetCode Link:

https://leetcode.com/problems/implement-queue-using-stacks/submissions/2143305986/

Implement a Queue using two stacks, s1 and s2, supporting the following operations:

- push(x) – Insert an element into the queue.
- pop() – Remove and return the front element.
- peek() – Return the front element without removing it.
- empty() – Check whether the queue is empty.

Algorithm:

- Create two stacks, s1 and s2.
- **Push:**
  - Insert the element into s1.
- **Pop:**
  - Move all elements from s1 to s2.
  - The top of s2 is the oldest element, so remove it.
  - Move the remaining elements back to s1.
- **Peek:**
  - Move all elements from s1 to s2.
  - The top of s2 is the front element.
  - Move all elements back to s1.
- **Empty:**
  - Return true if s1 is empty; otherwise return false.

Code:

```
#include <stack>
using namespace std;
class MyQueue {
private:
   stack<int> input;
   stack<int> output;
```

```
    void shiftStacks() {
        if (output.empty()) {
            while (!input.empty()) {
                output.push(input.top());
                input.pop();
            }
        }
    }

public:
    MyQueue() {

    }

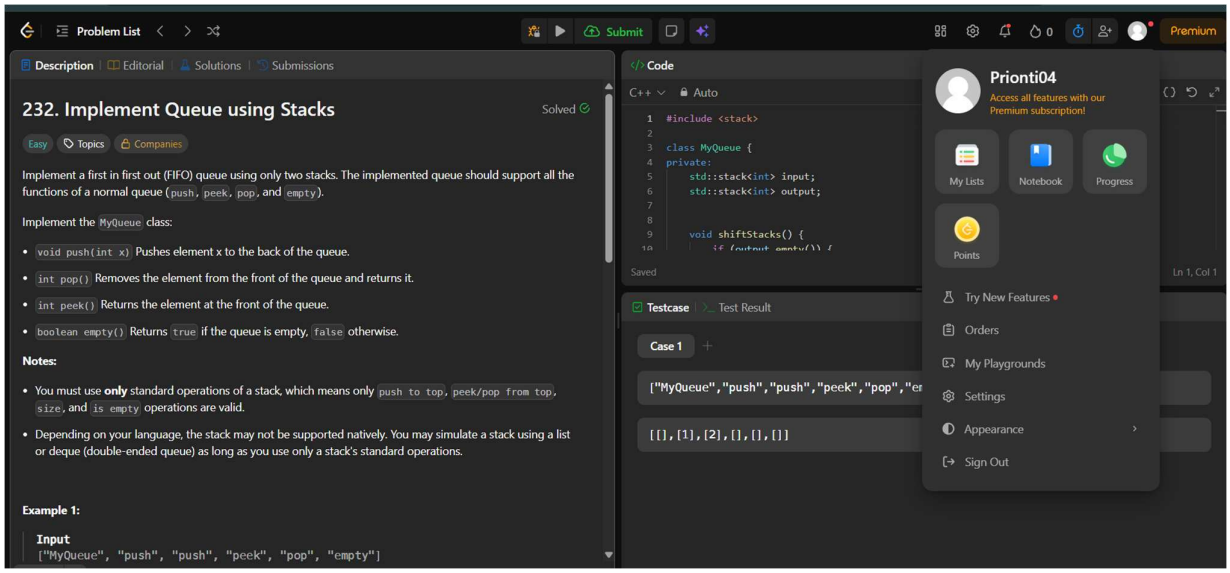

    void push(int x) {
        input.push(x);
    }


    int pop() {
        shiftStacks();
        int topElement = output.top();
        output.pop();
        return topElement;
    }


    int peek() {
        shiftStacks();
        return output.top();
    }


    bool empty() {
        return input.empty() && output.empty();
    }
};
```

Observations/Outcome: The program successfully implements a **FIFO Queue using two LIFO stacks** and supports push, pop, peek, and empty operations.



Time Complexity:

- **Push:** O(1)
- **Pop:** O(n)
- **Peek:** O(n)
- **Empty:** O(1)

Space Complexity: O(n).

Learning outcomes:

1. Learned how to implement a **Queue using Stacks**.
2. Understood the difference between **FIFO and LIFO**.
3. Learned how two stacks can be used to reverse element order.
4. Understood the time and space complexity of different queue operations.

Experiment 3.2

Aim: To implement a C++ program to find the **largest rectangular area** that can be formed in a histogram using a stack.

Objective:

- To understand the use of a **stack** for histogram problems.
- To find the nearest smaller elements efficiently.
- To calculate the maximum rectangular area in linear time.

Problem Statement / LeetCode Link:

https://leetcode.com/problems/largest-rectangle-in-histogram/submissions/2143412797/

Given an array heights representing the heights of bars in a histogram, find the area of the **largest rectangle** that can be formed using one or more consecutive bars.

Algorithm:

- Create an empty stack st to store indices of bars.
- Traverse the histogram from left to right.
- Treat the height after the last bar as 0 to ensure all remaining bars are processed.
- While the current height is smaller than the height at the stack's top:
  - Remove the top index from the stack.
  - Calculate its height and possible width.
  - Calculate the rectangle area.
  - Update maxArea if the calculated area is larger.
- Push the current index into the stack.
- After processing all bars, return maxArea.

Code:

```
#include <vector>
#include <stack>
using namespace std;

class Solution {
public:
   int largestRectangleArea(vector<int>& heights) {
      stack<int> st;
      int maxArea = 0;
      int n = heights.size();

      for (int i = 0; i <= n; i++) {

         int currentHeight = (i == n) ? 0 : heights[i];


         while (!st.empty() && currentHeight < heights[st.top()]) {
            int height = heights[st.top()];
            st.pop();

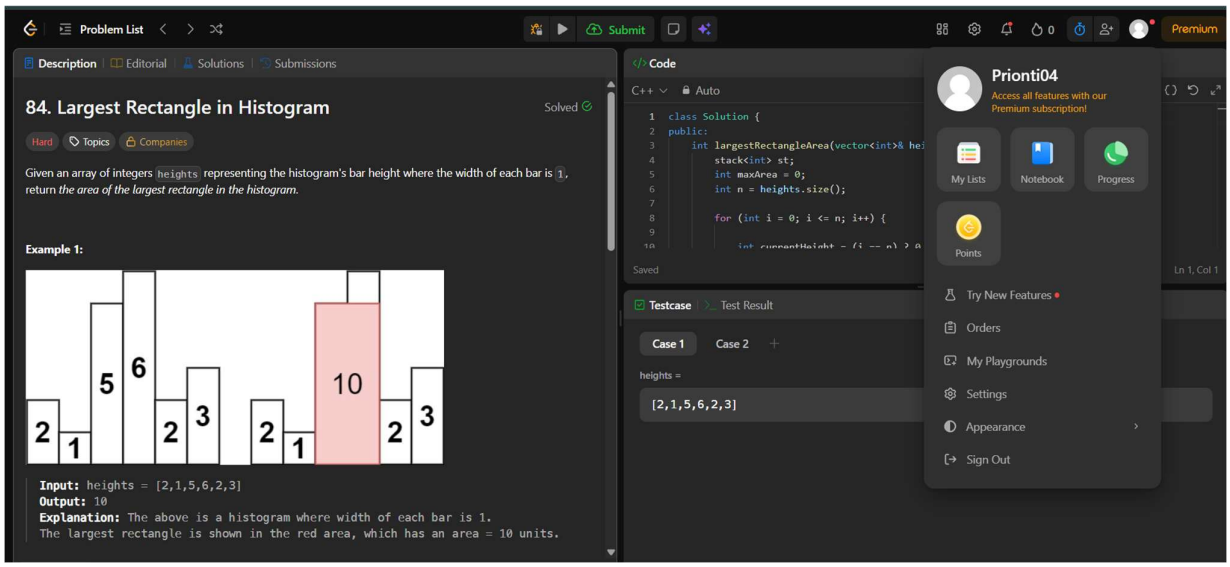

            int width = st.empty() ? i : i - st.top() - 1;

            maxArea = max(maxArea, height * width);
         }
```

```
            st.push(i);
        }

        return maxArea;
    }
};
```

Observations/Outcome: The program successfully finds the **maximum rectangular area** in a histogram using a monotonic stack.



Time Complexity: **O(n)** Each index is pushed and popped from the stack at most once.

Space Complexity: **O(n)** — The stack can contain up to n indices.

Learning outcomes:

1. Learned how to use a **monotonic stack**.
2. Learned how to calculate the width and area of possible rectangles.
3. Understood how to solve the histogram problem efficiently in **O(n)** time.
4. Learned how stack-based solutions can optimize problems that otherwise require nested loops.